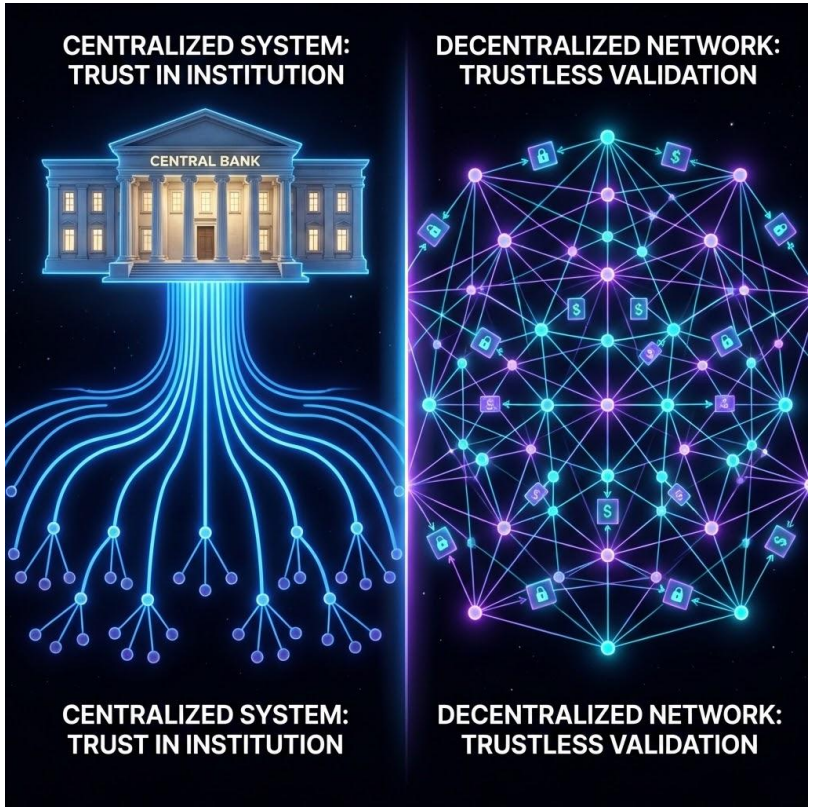
Blockchain and Cryptocurrencies Technologies

Crypto Primitives for Distributed Ledgers

Crypto Primitives for Distributed Ledgers

- Understanding the cryptographic foundations of trustless systems
- From mathematical primitives to real-world decentralized systems

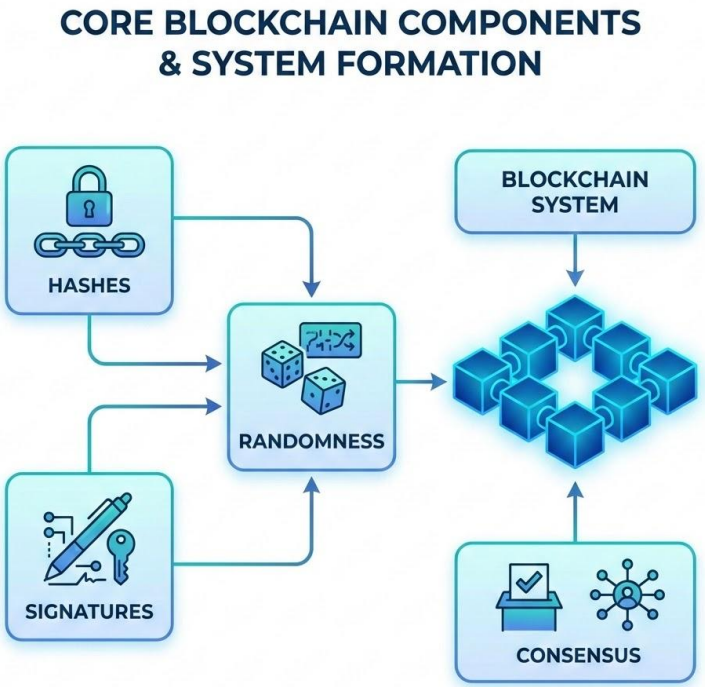
How Do We Build Trust Without a Central Authority?



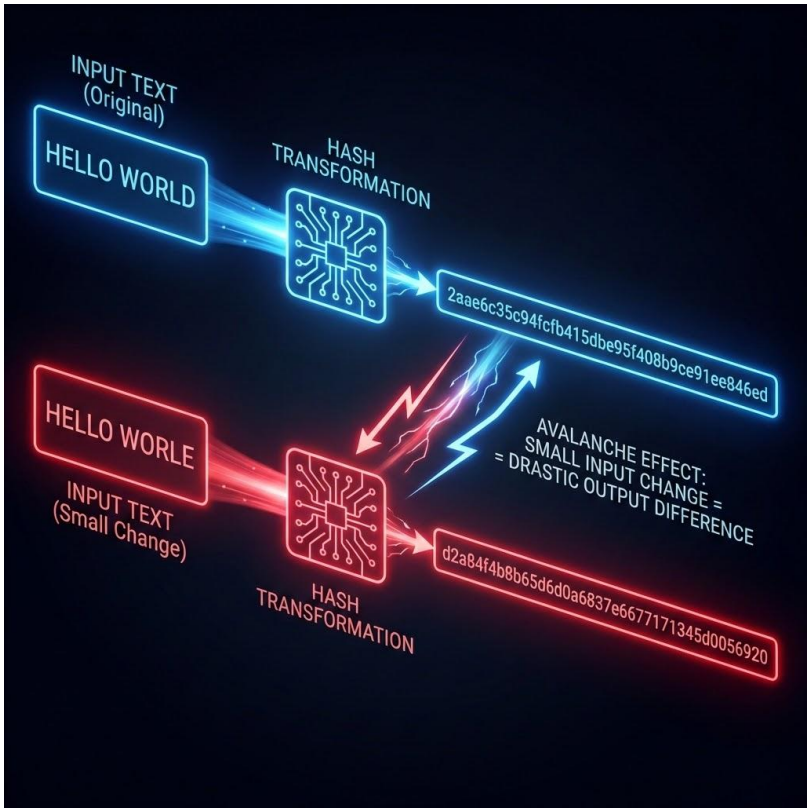
- Traditional systems rely on trusted intermediaries (banks, governments, registries)
- These intermediaries maintain:
  - Identity
  - transaction validation
  - dispute resolution
- Distributed systems remove the central authority
- Problem: how do we ensure correctness, integrity, and ownership?
- This is fundamentally a cryptographic problem

The Foundations of Blockchain Systems

- Distributed ledgers are built on a small set of primitives:
  - Cryptographic Hash Functions
  - Digital Signatures
  - Secure Randomness
  - Consensus Mechanisms
- These primitives combine to:
  - ensure integrity
  - enforce ownership
  - prevent manipulation
- Without these primitives, blockchain cannot exist



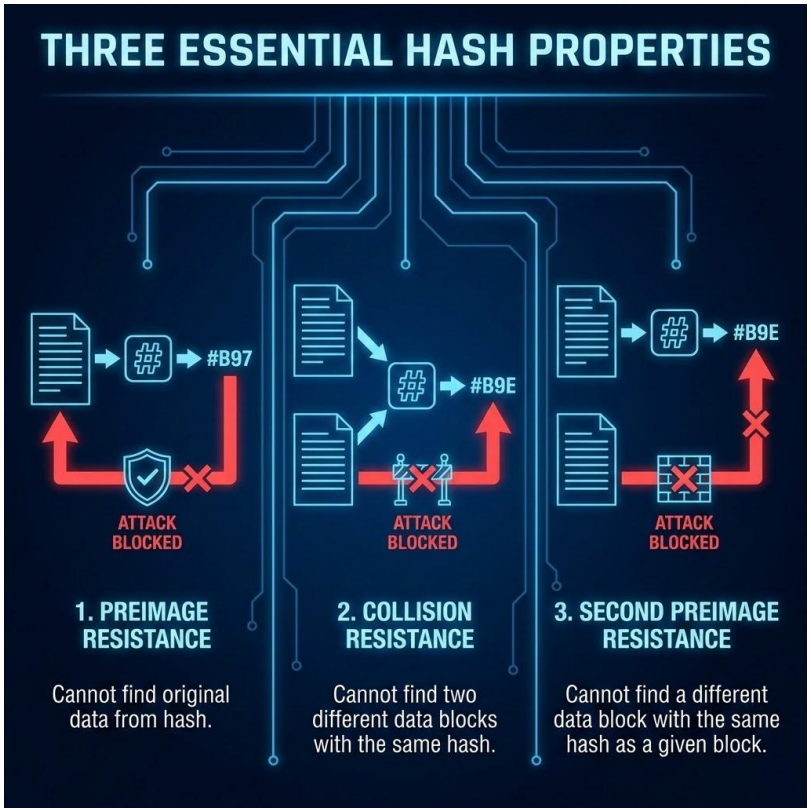
Hash Functions: The Backbone of Integrity



- A hash function maps arbitrary input → fixed-size output
- Properties:
  - Deterministic
  - fast to compute
  - infeasible to invert
- Small input change → completely different output (avalanche effect)
- Used to:
  - ensure data integrity
  - link blocks in a chain
- Example: SHA-256 in Bitcoin

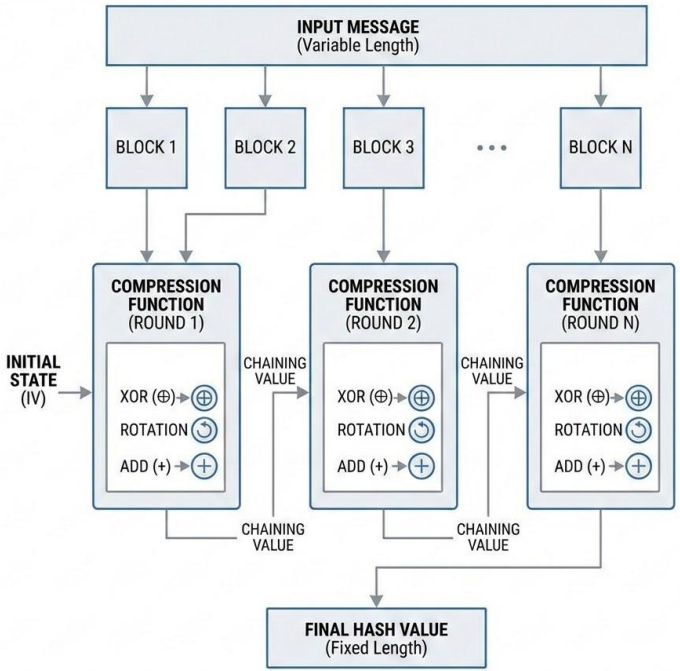
Why Hashes Are Secure

- Pre-image resistance:
  - given hash, cannot find original input
- Second pre-image resistance:
  - cannot find another input with same hash
- Collision resistance:
  - infeasible to find two inputs with same hash
- These properties guarantee:
  - tamper detection
  - structural integrity
- Any modification breaks the chain

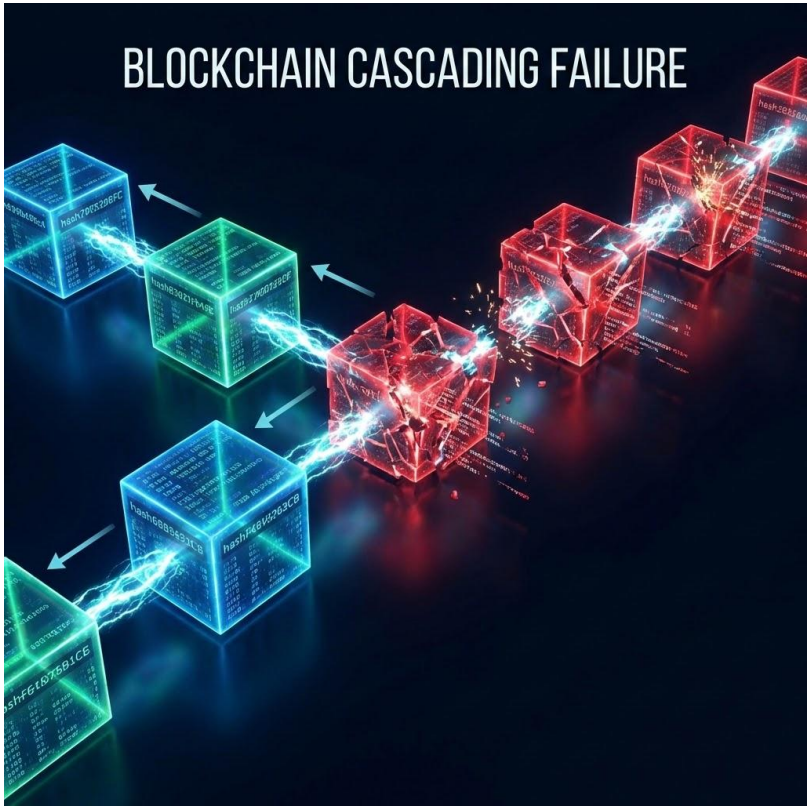


Inside a Cryptographic Hash Function

- Hash functions are built from iterative compression functions
- Input is processed in fixed-size blocks (e.g., 512 bits in SHA-256)
- Each block updates an internal state:
  - chaining value
- Final output depends on:
  - all previous blocks
- Uses:
  - bitwise operations (XOR, rotations, shifts)
  - modular arithmetic
- Designed to:
  - maximize diffusion and confusion



Chaining Blocks Through Hashes



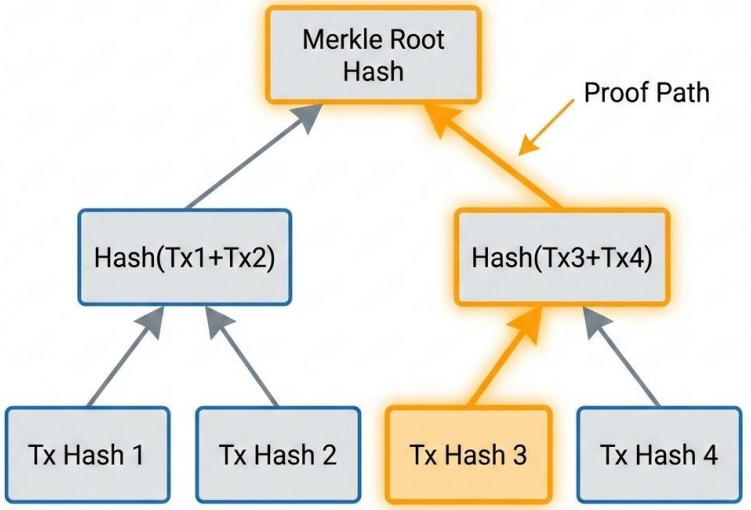
- Each block contains:
  - transaction data
  - Timestamp
  - previous block hash
- This creates a chain:
  - block n references block n-1
- Changing one block:
  - invalidates all following blocks
- Ensures immutability of ledger

Hash Functions in Mining

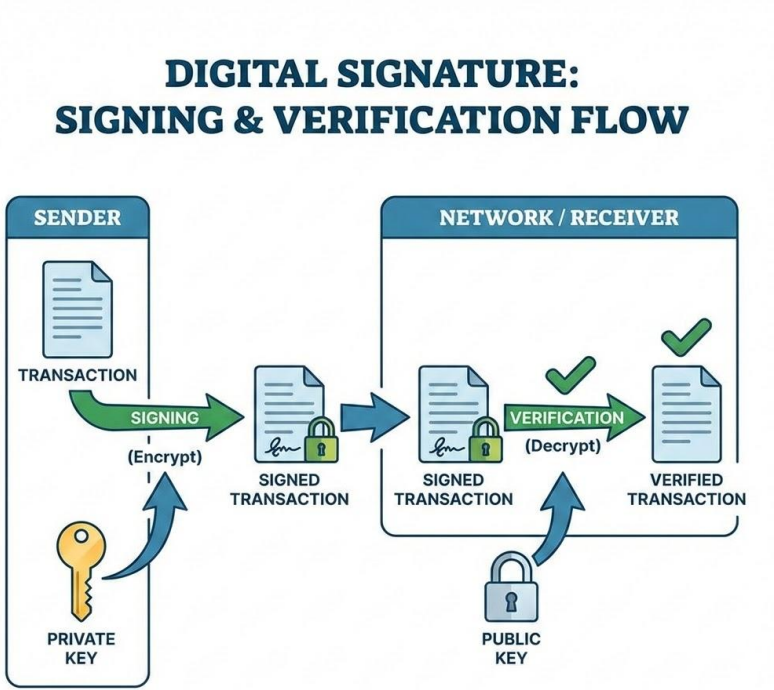
- Proof of Work = computational puzzle
- Goal:
  - find hash below target threshold
- Achieved through brute force
- Security comes from:
  - computational cost
- Mining = searching for partial hash collisions

Merkle Trees: Efficient Integrity Verification

- Transactions are not stored linearly for verification
- Instead, they are organized into a Merkle Tree
- Leaves:
  - hashes of transactions
- Internal nodes:
  - hash of children
- Root hash summarizes entire dataset
- Allows:
  - efficient inclusion proofs (Merkle proofs)
- Verification complexity:
  - O(log n) instead of O(n)

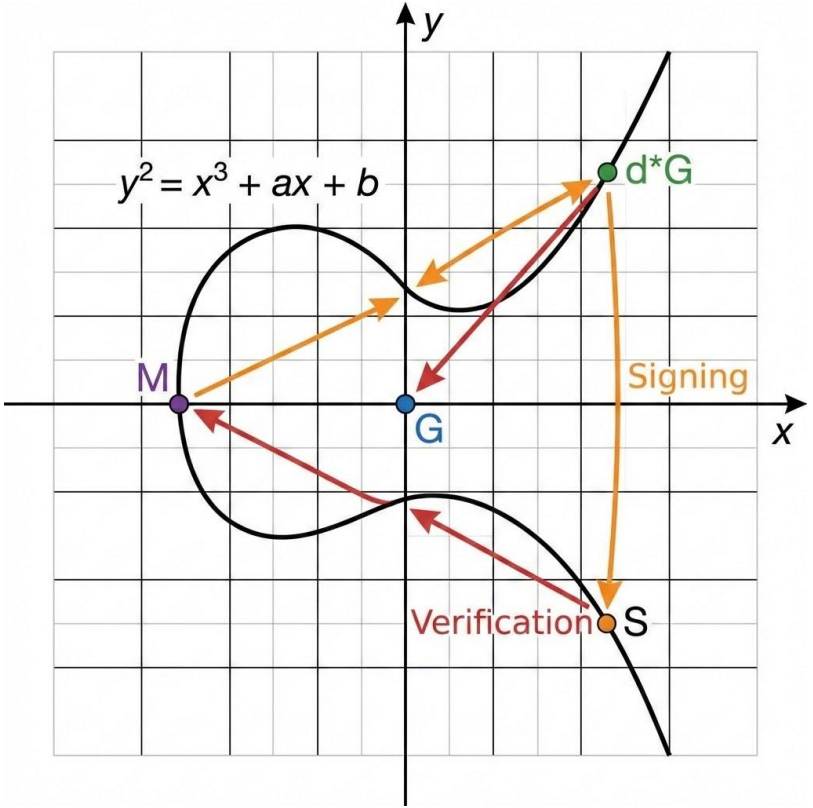


Ownership Without Identity



- Digital signatures enable:
  - Authentication
  - Integrity
  - non-repudiation
- Based on asymmetric cryptography:
  - private key signs
  - public key verifies
- No need for identity provider
- Control = possession of private key

How Digital Signatures Actually Work (ECDSA)

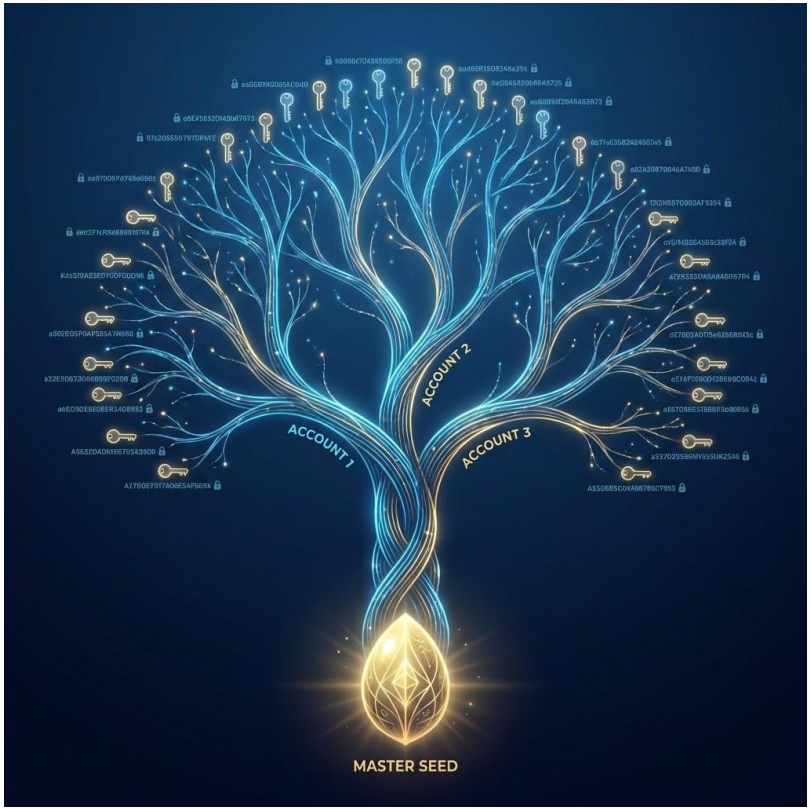


- Based on elliptic curve cryptography (e.g., secp256k1 in Bitcoin)
- Private key:
  - random number
- Public key:
  - point on elliptic curve
- Signing process:
  - uses private key + message hash
- Verification:
  - checks mathematical consistency
- Security relies on:
  - Elliptic Curve Discrete Logarithm Problem

Wallets Are Not Accounts

- Wallet = collection of key pairs
- Public key:
  - used as address
- Private key:
  - grants control over funds
- No central recovery mechanism
- Loss of private key = irreversible loss

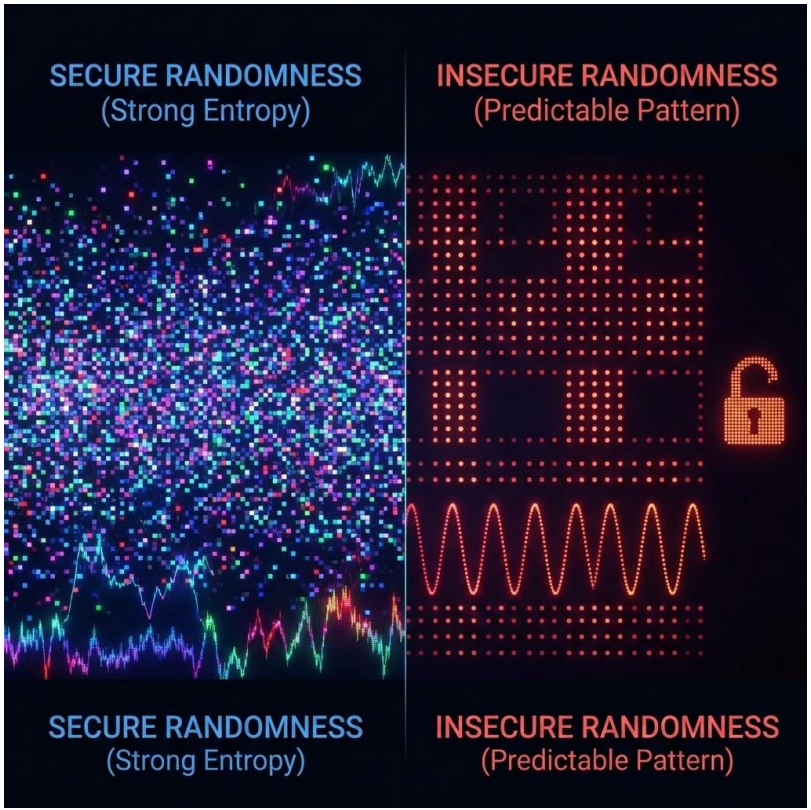
From One Seed to Many Keys



- Deterministic wallets generate keys from a seed
- Seed phrase = root of all keys
- Benefits:
  - easy backup
  - hierarchical key generation
- Standards:
  - BIP32, BIP39
- Single failure point: seed compromise

Randomness: The Weakest Link

- Cryptographic systems depend on randomness
- Weak randomness → predictable keys
- Sources:
  - PRNG vs TRNG
- Bad randomness leads to:
  - key compromise
  - signature attacks
- Often the most overlooked vulnerability



When Randomness Fails

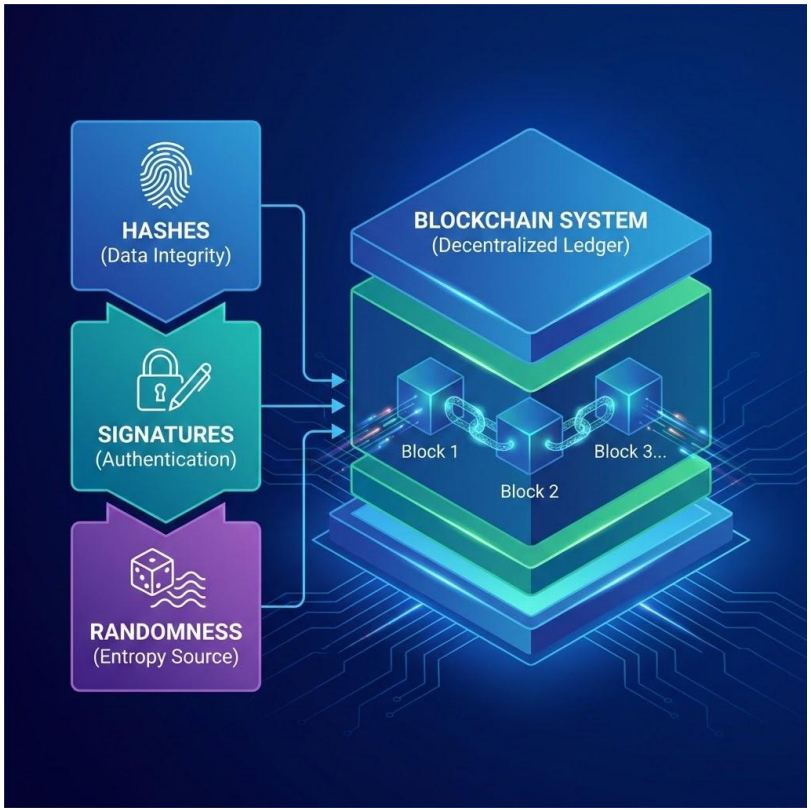
- Systems have failed due to weak entropy
- Example:
  - predictable seeds based on timestamps
- Attackers can:
  - brute-force key space
- Lesson:
  - randomness is critical infrastructure

When Randomness Breaks Everything

- Poor entropy leads to predictable private keys
- Example failures:
  - reused nonce in ECDSA
  - weak random generators
- Consequence:
  - attackers recover private key
- Famous cases:
  - Bitcoin wallet key leaks
  - Sony PlayStation ECDSA failure
- Lesson:
  - randomness must be:
    - Unpredictable
    - high entropy
    - properly seeded

How Primitives Build Blockchain

- Hash → integrity
- Signature → ownership
- Randomness → security
- Consensus → agreement
- Together they create:
  - decentralized trust
  - tamper resistance
  - permissionless systems



Beyond Signatures: Proving Without Revealing

- Traditional crypto:
  - prove by revealing data
- Zero-Knowledge Proofs:
  - prove without revealing
- Example:
  - prove age > 18 without showing birthdate
- Not required for basic blockchain
- Critical for privacy-preserving systems

The Essence of Distributed Trust

- Blockchain is not a single technology
- It is a composition of primitives
- Each primitive solves a specific problem:
  - integrity, ownership, randomness, agreement
- Security depends on weakest component
- Understanding primitives = understanding blockchain